

Calcolatori Elettronici

27 febbraio 2026

Teoria

Esercizio 1 (6 punti): Codificare la parola binaria `10111` utilizzando un codice di Hamming con $h = 3$, utilizzando bit di parità pari, mostrando tutti i passaggi. Si supponga di ricevere il valore `111001111`: illustrare quali passaggi è necessario effettuare per determinare se c'è stato o meno un errore di trasmissione e ricostruire, se possibile, il valore originale.

Esercizio 2 (10 punti): Si realizzi una rete sequenziale in grado di pilotare una coppia di spie luminose. La rete ha due uscite (A , B) che indicano, rispettivamente, lo stato di accensione (1) o spegnimento (0) di ciascuna delle due spie. La rete ha in ingresso un segnale x che indica la modalità di lampeggiamento delle spie: se $x = 1$, le uscite operano in fase, alternando $(0,0) \leftrightarrow (1,1)$; se $x = 0$ le uscite operano in contrapposizione di fase, alternando $(1,0) \leftrightarrow (0,1)$. Le uscite A e B devono essere funzioni esclusive dello stato interno per prevenire instabilità durante la transizione dell'ingresso x . Si realizzi il circuito equivalente dell'intera rete.

Esercizio 3 (9 punti): Si consideri il seguente frammento in assembly:

```
1  mystery:
2      movl    %edi, %eax
3      movl    %esi, %ecx
4      testl   %eax, %eax
5      jnz     .skip1
6      movl    $0, %eax
7      ret
8  .skip1
9      cmpl    %eax, %ecx
10     jnc     .skip2
11     xorl    %ecx, %eax
12     andl    $0xFFFFFFFF, %eax
13     ret
14  .skip2:
15     addl    %ecx, %eax
16     shrl    $1, %eax
17     ret
```

Convertire il frammento nel suo equivalente C, dettagliando le motivazioni per la conversione effettuata.

Esercizio 4 (5 punti): Illustrare la necessità dell'operazione di refresh nelle memorie DRAM, discutendone l'impatto sulle prestazioni del sistema.

Soluzioni

Esercizio 1

Per procedere alla codifica della parola fornita, è necessario determinare il numero di bit di parità richiesti affinché la condizione $m + 1 \leq 2^p - p$ sia soddisfatta. Data una lunghezza del messaggio $m = 5$, il valore minimo di p che soddisfa la disequazione è $p = 4$. I bit di parità devono essere collocati nelle posizioni corrispondenti alle potenze di due, ovvero nelle posizioni 1, 2, 4, 8. La determinazione dei valori di parità avviene analizzando la copertura di ciascun bit di controllo sulle posizioni la cui decomposizione binaria include la potenza di due specifica:

	p_1	p_2	1	p_4	0	1	1	p_8	1	Risultato
p_1	✓		✓		✓		✓		✓	1
p_2		✓	✓			✓	✓			1
p_4				✓	✓	✓	✓			0
p_8								✓	✓	1

Al termine del calcolo, si ottiene la parola codificata **111001111**. Considerando la ricezione della sequenza **111001111**, l'analisi dell'integrità del dato avviene attraverso il ricalcolo dei bit di controllo sulla stringa ricevuta:

- Bit di controllo P_1 : posizioni 1, 3, 5, 7, 9; i bit 1, 1, 0, 1, 1, hanno somma modulo 2 pari a 0: corretto.
- Bit di controllo P_2 : posizioni 2, 3, 6, 7; i bit 1, 1, 1, 1, hanno somma modulo 2 pari a 0: corretto.
- Bit di controllo P_4 : posizioni 4, 5, 6, 7; i bit 0, 0, 1, 1, hanno somma modulo 2 pari a 0: corretto.
- Bit di controllo P_8 : posizioni 8, 9; i bit 1, 1, hanno somma modulo 2 pari a 0: corretto.

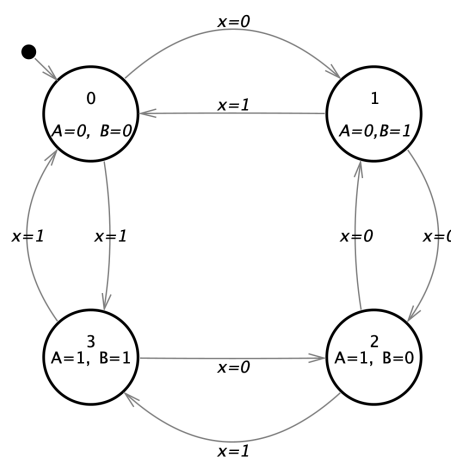
Poiché tutti i controlli di parità hanno restituito esito negativo per la presenza di errori, si conclude che la trasmissione è avvenuta correttamente e il dato originale può essere estratto rimuovendo i bit nelle posizioni di controllo.

Eliminando i bit di parità situati nelle posizioni corrispondenti alle potenze di due, si ricostruisce il valore originale, che risulta essere **10111**.

Esercizio 2

Dal momento che l'esercizio richiede che le uscite A e B debbano essere funzioni esclusive dello stato interno, si progetta una macchina di Moore.

L'automa che risolve l'esercizio prevede la presenza di una coppia di stati per ciascuna modalità di lampeggiamento. Una coppia corrisponde alle transizioni $(0,0) \leftrightarrow (1,1)$, mentre l'altra coppia corrisponde alle transizioni $(1,0) \leftrightarrow (0,1)$. Lo stato iniziale viene associato a quello stato in cui entrambe le spie luminose sono spente. Quando il valore di x non cambia, la macchina oscilla nella stessa coppia di stati. Quando il valore di x cambia, la macchina transita nell'altra coppia di stati. Ha senso utilizzare 0 come stato iniziale, dal momento che all'accensione i bit di stato varranno entrambi 0. L'automa corrispondente può essere il seguente:



Le codifiche sono già tutte date. Si può procedere quindi alla compilazione della tabella degli stati e delle transizioni:

y_1	y_0	x	y'_1	y'_0	A	B
0	0	0	0	1	0	0
0	0	1	1	1	0	0
0	1	0	1	0	0	1
0	1	1	0	0	0	1
1	0	0	0	1	1	0
1	0	1	1	1	1	0
1	1	0	1	0	1	1
1	1	1	0	0	1	1

Da qui, per progettare il circuito, occorre minimizzare le equazioni di eccitazione di y'_1 ed y'_0 , così come le equazioni per il calcolo di A e B . Questo può essere fatto utilizzando le mappe di Karnaugh.

Minimizzazione della variabile y'_1 :

$y_0 \backslash y_1$	00	01	11	10
0	0	1	0	1
1	0	1	0	1

da cui $y'_1 = y_0\bar{x} + \bar{y}_0x$.

Minimizzazione della variabile y'_0 :

$y_0 \backslash y_1$	00	01	11	10
0	1	1	0	0
1	1	1	0	0

da cui $y'_0 = \bar{y}_0$.

Minimizzazione della variabile A :

$y_0 \backslash y_1$	00	01	11	10
0	0	0	0	0
1	1	1	1	1

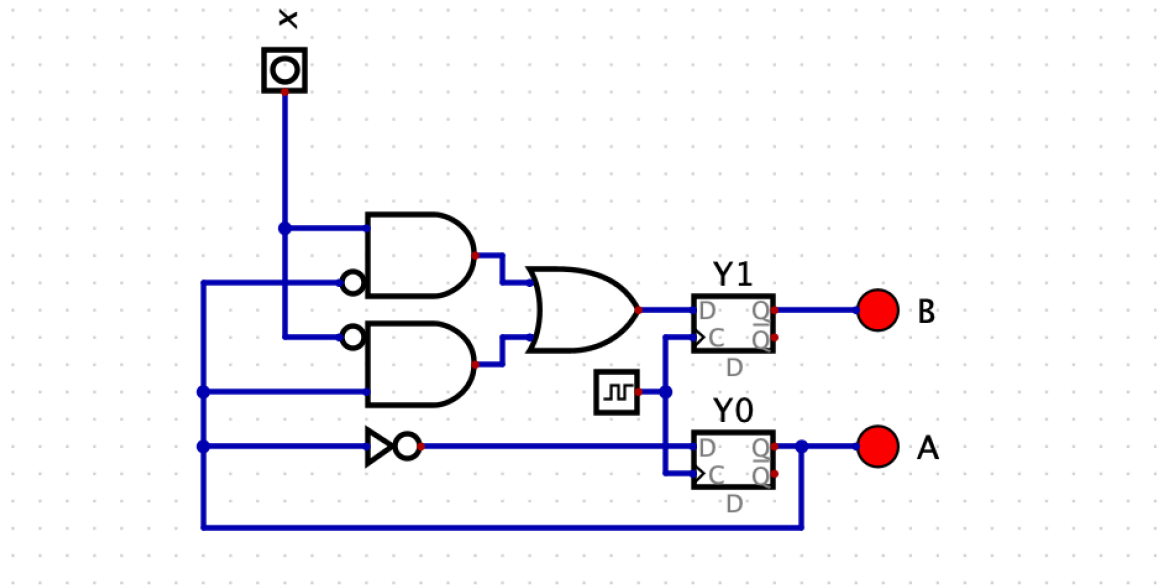
da cui $A = y_1$.

Minimizzazione della variabile B :

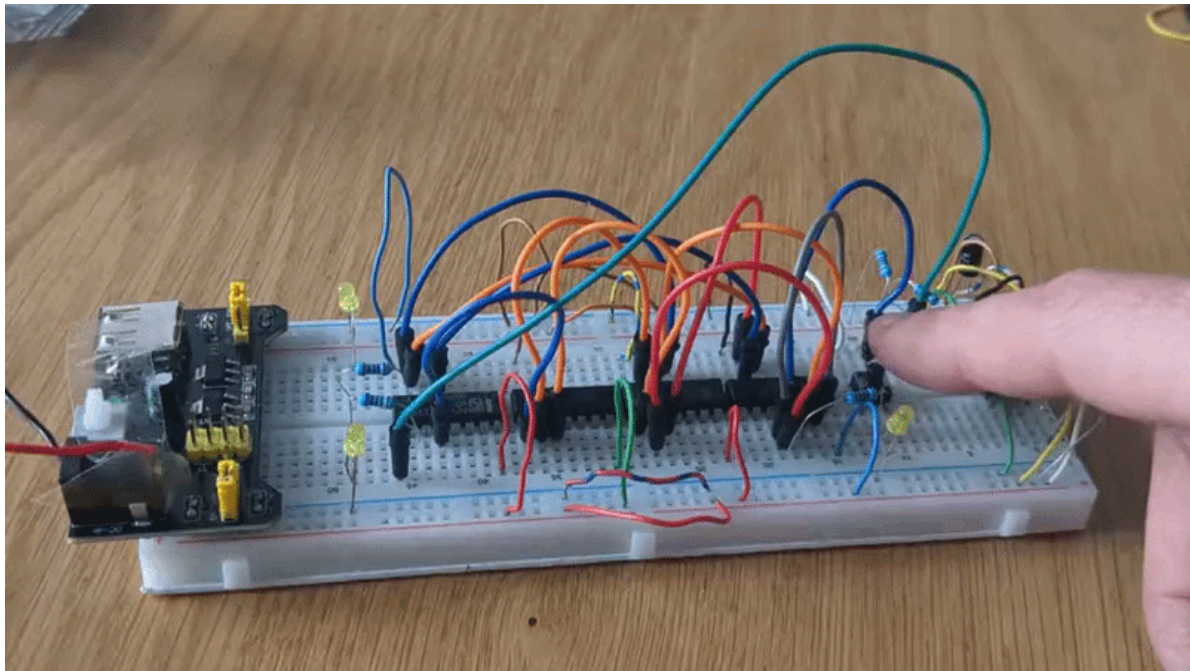
y_0x y_1	00	01	11	10
0	0	0	1	1
1	0	0	1	1

da cui $B = y_0$.

Pertanto, il circuito equivalente della macchina sequenziale è:



Il circuito è talmente semplice che può anche essere realizzato fisicamente (il video è allegato alla soluzione):



Esercizio 3

L'uso dei registri `%edi` e `%esi` indica che la funzione accetta due parametri interi a 32 bit. Poiché viene fatto un confronto utilizzando `jnc`, i tipi di dato devono essere considerati interi senza segno (`unsigned int`). Il registro `%eax` viene utilizzato per memorizzare il valore di ritorno della funzione. La firma della funzione in C dovrebbe essere la seguente: `unsigned int mystery_function(unsigned int a, unsigned int b);`

Il corpo della funzione implementa una struttura di controllo condizionale che gestisce tre casi distinti: quando il primo parametro è zero, quando il primo parametro è minore del secondo, e il caso rimanente. In ciascuno dei tre casi vengono calcolati dei valori di ritorno differenti, poiché viene scritto il registro `%eax` prima dell'istruzione `ret`. Il codice corrispondente in C sarà quindi:

```
1 unsigned int mystery(unsigned int a, unsigned int b)
2 {
3     if (a == 0)
4         return 0;
5     if (b >= a)
6         return (a + b) >> 1;
7     return (a ^ b) & 0xFFFFFEE;
8 }
```

Progetto

Un processore `z64` controlla una termocamera a infrarossi, composta dalle periferiche `PULSANTE`, `IR` e `SCHERMO`. Alla pressione di `PULSANTE`, il processore acquisisce dalla periferica `IR` una matrice di 800×600 word, indicanti ciascuna un valore di temperatura. Tale matrice viene acquisita in modalità sincrona, un punto per volta, interrogando ciclicamente la periferica `IR`. La matrice viene memorizzata in memoria in un buffer appositamente dedicato. Il valore minimo di temperatura restituito dal sensore è 0°C , il valore massimo è 80°C .

Al termine dell'acquisizione, il processore converte la matrice di word in un'immagine RGB di uguale dimensione, mantenuta in un altro apposito buffer, associando il colore blu ad una temperatura fredda, il colore giallo ad una temperatura tiepida, il colore rosso ad una temperatura calda. Ciascun pixel dell'immagine è memorizzato in una longword, in cui i tre byte meno significativi rappresentano, rispettivamente, la componente R, la componente G e la componente B. L'assegnazione di un colore ad una temperatura rilevata segue la seguente tabella di conversione.

Intervallo Temperatura	Colore Dominante	R	G	B
0 - 26	Freddo (Blu)	0	0	255
27 - 53	Tiepido (Giallo)	255	255	0
54 - 80	Caldo (Rosso)	255	0	0

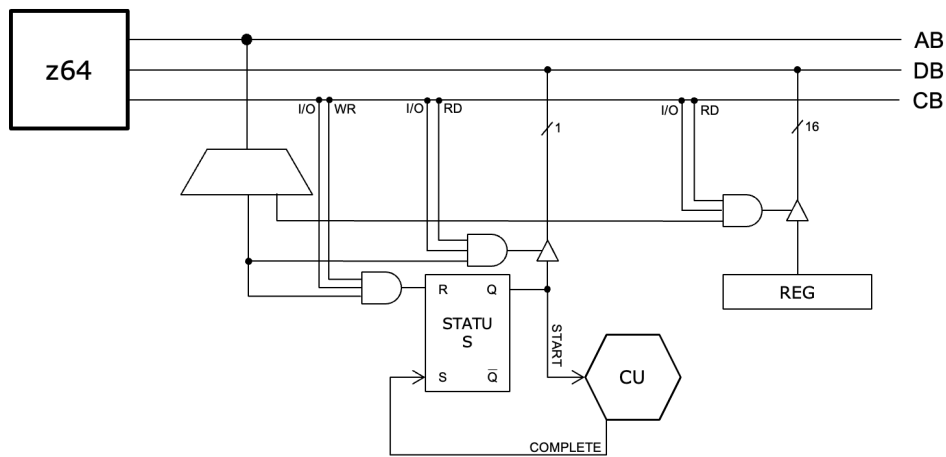
Al termine della conversione, l'immagine viene inviata al dispositivo `SCHERMO` utilizzando il DMAC di sistema. `SCHERMO` è collegata unicamente al DMAC di sistema.

Realizzare:

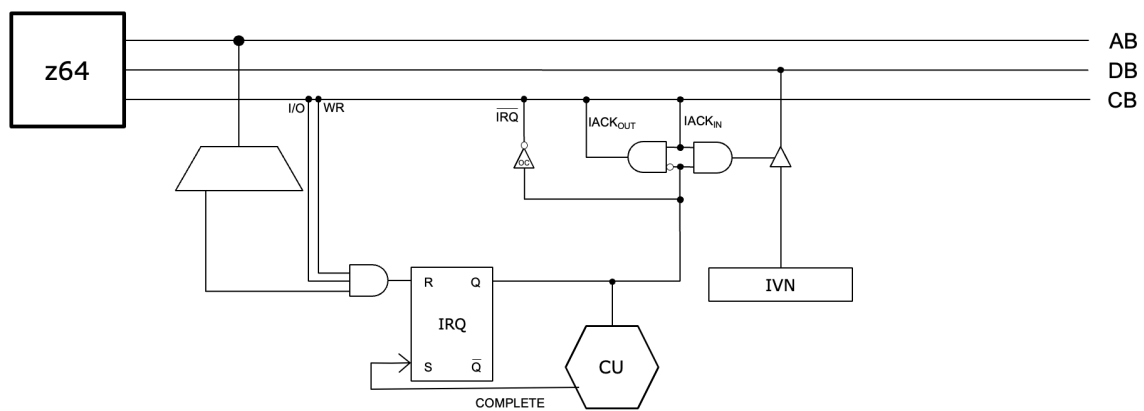
- Le interfacce dei dispositivi `PULSANTE` e `IR`;
- Tutto codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

La periferica `IR` è un dispositivo sincrono di input, pertanto la sua interfaccia è la seguente:



La periferica **PULSANTE** è invece un dispositivo asincrono, ma che non produce né consuma dati:



Il codice del sistema può essere il seguente.

```

1  .org 0x800
2  .equ PULSANTE_IRQ, 0x0100
3  .equ IR_STATUS, 0x0102
4  .equ IR_DATA, 0x0104
5  .equ SCHERMO, 0x0200
6  .equ PIXEL_TOT, 480000
7
8  .data
9  flag_pulsante: .byte 0
10 buffer_temp: .fill PIXEL_TOT, 2, 0
11 buffer_rgb: .fill PIXEL_TOT, 4, 0
12
13 .text
14 main:
15     sti
16     .wait:
17     cmpb $0, flag_pulsante
18     jz .wait
19
20     movb $0, flag_pulsante
21     call acquisizione
22     call conversione
23
24     # Trasferimento DMA
25     movq $buffer_rgb, %rsi
26     movq $PIXEL_TOT, %rcx
27     movw $SCHERMO, %dx
28     cld
29     outsl
30

```

```

31     jmp .wait
32
33 acquisizione:
34     leaq buffer_temp, %rdi
35 .loop:
36     outb %al, $IR_STATUS    # Trigger lettura
37 .bw:
38     inb $IR_STATUS, %al
39     testb $1, %al
40     jz .bw
41
42     inw $IR_DATA, %ax
43     movw %ax, (%rdi)
44     addq $2, %rdi
45     cmpq $buffer_temp + PIXEL_TOT * 2, %rdi
46     jnz .loop
47     ret
48
49 conversione:
50     xorq %rcx, %rcx
51 .loop2:
52     movw buffer_temp(, %rcx, 2), %ax
53     movl $0x000000FF, %edx    # Ipotizza FREDDO (Blu)
54     cmpw $27, %ax
55     jc .store
56     movl $0x00FFFF00, %edx    # Ipotizza TIEPIDO (Giallo)
57     cmpw $54, %ax
58     jc .store
59     movl $0x00FF0000, %edx    # Altrimenti è CALDO (Rosso)
60 .store:
61     movl %edx, buffer_rgb(, %rcx, 4)
62     addq $1, %rcx
63     cmpq $PIXEL_TOT, %rcx
64     jnz .loop2
65     ret
66
67 .driver 0 # PULSANTE
68     movb $1, flag_pulsante
69     outb %al, $PULSANTE_IRQ
70     iret

```